

Bantam: untermenge Java

int, boolean

Object, String, TextIO, Sys

Arrays

int[] n = new int[10];

typecast

(targetclass) (expr);

expr instanceof targetclass

Programmausführung Main.main()

class A extends B {

int a = 1;

boolean b;

void f(int a, ...) { kein Überladen

return; // 1 an Ende

}

je nur 1mal!
name.name[index] =

Attributzugriff nur über this, super

keine Deklaration in for

MIPS 32 Risc, Load-Store-Architektur

\$zero

\$s0 this-Zeiger

\$r0 return val

\$a0 Empfängerobjekt Methodenaufruf

\$a1 Zeilennummer

\$a2 Dateiname

\$t0-\$t9 temp Var

\$s0-\$s7 gesicherte

\$sp Stack Pointer

\$fp

\$ra Return Address

Parameter über Stack

A:

.word

.ascii

.byte

.align n

val

n

2^n

store load

si li \$t0 42

sw lw \$t0 4(\$t0)

sa la \$t0 table

intermediate
word
address

jump

jnl label and (int) (\$ra)

~~jr~~

jr \$ra register

branch

bne \$t0 \$t1 label not equal

lt

...

arith

add, sub, mul

sll \$t0 \$t1 2 leftshift logical

move \$t0 \$t1 \$t0 + \$t1

sll \$r \$a 2 // 4 Byte

bne \$r \$zero label // check null pointer

Dynamic Dispatch

Polymorphe Methodenaufrufe

direct static - falls Optimierung möglich

indirekt dynamisch
Aufruf ohne Vererbung

dynamische Prüfung
null-pointer-error
bne \$a0 \$zero ~~latet~~
jal null-pointer-error

Assembler Zieladdr. in Register
jal r \$t0
└─ register
└─ and link
└─ jump

Instanz ~ Typinformation
└─ z.B. Type-ID
Type-ID ~ vtable

vtable - virtual Method Table

~~Method-ID~~ Method-ID ~ Startaddr. Routine

- 3 Speicherzugriffe

verteilt ⇒ Addr. vtable in Instanzen
2 Speicherzugriffe

⇒ vtable in Instanz
- Speicher
- Vererbung
- Speicherlayout
- Cache

Vererbung einfach, kein Func. entfernen

Erweitern Ende einfügen
Verändern (ind. super)
└─ in Oberkl. vtable

⇒ Anpassen vtable erstmalig Kopie

Erzeugen von Objekten

neuer Speicher syscall
init Attr. mit Standardwerten, Type-ID, Addr. vtable
└─ Size in Bytes
└─ für clone

Template Klasse-template
Object.clone
Klasse-init
└─ init Oberklasse

la \$a0 X-template
jal Object.clone // auf template
move \$a0 \$v0
jal X-init // auf Kopie

Sonderfall String-Literale

als Singleton
vorkommende Zeichenkette - 1 String-Objekt
- template enthält bereits Zeichen
Variable Größe String Länge
Ascii Null als Endmarker
Auffüllen auf Wortgrenze

templ. für
Gedächtnis

Type-ID Klasse String
Gesamtgröße in Byte
V-table Kl. String
String Länge ohne Null
• ascii Zeichen
• byte 0 Endmarker
• align 2 Fülle auf Vielfachen 2¹²

dynamische Strings von Stringtemplate

Methodenprotokoll

Übergabe von Parametern

auf Stack

links nach rechts

aufgerufenen räumt auf

\$a0: this-Zeiger Empfängerobj.

\$v0: Rückgabewert

Aufgerufenen

li \$r0

sw \$r0

sub \$sp

...

li \$a1

li \$a2

lw \$a0

bne \$a0

jnl

label:

lw \$t0

lw \$t0

jnlr \$t0

sw \$r0

Parameter auf Stack

Zeile

Str. Dateiname

// Zeiger Instanz

Label

-null pointer error

// vtable

// Addr. Routine

// indirekter Sprung

// Rückgabe in lokale var

Ausnahmebehandlung bei ungültigem Empf. obj.

\$a1 Zeilennummer

\$a2 template string literal

anell dates

Aufgerufenen

add \$sp \$sp, -16

sw \$ra 12(\$sp) // Ret. Addr.

sw \$fp 8(\$sp) // Frame pointer alt

sw \$s0 4(\$sp) // this-Zeiger alt

move \$fp \$sp // aktueller fp

move \$s0 \$a0 // aktueller this-Zeiger

... \$r0

lw \$s0 4(\$sp) // Wiederherstellen

lw \$fp 8(\$sp) //

lw \$ra 12(\$sp) //

add \$sp \$sp, 24 // freigeben Frame & Parameter

jr \$ra // Rücksprung

Parameter

Ret. Addr.

FP alt

this alt

TypeID

size in Bytes

addr. vtable

Dynamische Typprüfung

Down cast

instance of

Zuweisung Array-Element

+TypeID für TypID array

TypeID nach Pre-Order DFS

k Elemente in Unterbaum

L subclass.num-table

A konform zu B

$$typeid(B) \leq typeid(A) \leq typeid(B) + k$$

class.name-table

TypeID $\rightarrow$ String literal class name

Wenn benutzt für

Fehlermeldung

CFG

BB - Anweisungen ohne Kontrollfluss
alle oder keine

Bankam Optimierung Middle End

Interface Optimization

setEntryBlock
Method Name
Verbose

CFG 2 SSA(),
PhiRemover(),

Basic Block analyze()
transform()

cfg. Basic Block

getNumOfEdges()

getNthOutEdge(int n)

check
print
printAll

List<TACInst> getInstructions

TAC - three addr. code

@l local var

@p par.

@f class attr.

Ref. to declaration

getLocation
setLocation

Dominanz $x \gg \text{dominiert } y \Leftrightarrow \forall \text{ Pfad } \theta \text{ zu } y \in x$

strikt $x \gg y \Leftrightarrow x \gg y \wedge x \neq y$

$\text{DOM}(u)$ = Menge Dominatoren

IDOM = immediate Dominator

Dominatorbaum

Dominatorgrenze $\overset{\text{Fringe}}{\text{DF}}(x)$

$\exists$ dominiert ter. pred. , aber nicht selbst

Finde merge points w

Füge w zu allen $\text{DF}(x)$ x zwischen w und $\text{IDOM}(u)$

Postdominanz $x \gg \text{postdominiert } y \Leftrightarrow \forall \text{ Pfade von } y \text{ zu Endknoten über } x$

Dominanz in reverse CFG

Kontrollabhängigkeit y von x kontrollabhängig $\Leftrightarrow$ Pfad zwischen x y

Dominatorgrenze in Reverse CFG

$\forall$ Knoten nach x von y postdominiert
 y postdominiert x nicht

cond x entscheide
ob y ausgeführt werden
muss

Def-Use-Graph Anweisung = node

auf SSA spärlicher

Datenflußanalyse

global - für ganze Prozedur
local - BB

Copy Propagation - verwende Originalvariable statt Kopie

ACP - available copies Menge (Zielvar, Originalvar)

global (v, w, block, position)

$$CP_{in}(b) = \bigcap_{d \in pred(b)} CP_{out}(d)$$

init $CP_{in}(Entry) = \emptyset$
 $b = V$

$$CP_{out}(b) = copy(b) \cup (CP_{in}(b) - KILL(b))$$

Constant Propagation - globales Const. Folding, ineffizient, kein GEN/KILL Schema

(var, val | $\perp$)
Variable

pessimistisch = default $\perp$
bei Konfluenzoperator $\wedge$ zwischen Blöcken

$$CONST(b) = \bigcap_{d \in preds(b)} Fd(CONST(d))$$

L transform BB

ein Pfad
V, succ
Rückwärts

Live Variables - Pfad zu Verwendung v ohne Def dazwischen

$$LiveOut(b) = \bigcup_{m \in succ(b)} UELVar(m) \cup (LiveOut(m) \cap \overline{VarKill(m)})$$

Upwards Exposed L def. var

V Pfad
N, preds
vorwärts

Available Expressions

$$Avail(b)$$

V, preds

Reaching Definitions - gelesen $\wedge$ auf Pfad $\neg$ def

$$Reaches(b) = \bigcup_{d \in preds(b)} (DEDef(d) \cup (Reaches(d) \cap \overline{DefKill(d)}))$$

Downwards exposed Definitions L überschriebene Def anderer Blöcke

N, succ

Very Busy Expr. $\forall$ succ evaluiert

$$VeryBusy(b) = \bigcap_{d \in succ(b)} (UEExpr(d) \cup VeryBusy(d) \cap \overline{ExprKill(d)})$$

Upwards exposed Expr. L Operanden überschrieben
in b vor überschreiben Operanden

Datenflussanalyse

iteratives Verfahren GEN/KILL

Terminierung Menge wächst/schrumpft monoton
↳ zu Limit

Korrektheit über Pfad, Verbandalgebra

Effizienz

Abarbeitungsreihenfolge

Prioritätswarteschlange
↳ überwinden

Reverse Post-Order

Tiefensuche unbesuchter prerer
Nummer bei Rückweg

abspeichern in Permutation

Worklist

füge von dir abhängige zu Worklist hinzu

- Annahme alle BB können ausgeführt werden

Arrayzugriffe nur gesamtes

Zeiger

Addressarithmetik
Typisierung

worst case
Annahme

Prozeduranruf

var-Parameter
nicht lokale Var.
globale
Zeiger → alles

+ einmal Implementieren Data Flow Framework

Static Single Assignment - Form

Wertinstanzen ursprünglicher Variablen

Phi-Funktionen an Merge-Points

Kontrollzweig - Parameter

+ keine Wert Auslöschung

+ textuelle eq

- Arrays

- Prozeduraufrufe

Transformieren

in SSA join Knoten vorgegeben; traversieren AST

Strukturierte Programmiersprache

in BB höchster aktueller Wert

gesamten CFG in if $\Rightarrow$ immer irgendwann join

bei branch

neuer uneingehängelter join-Knoten

bei Zuweisung:

sichere Wertinstanz von split Knoten

[Einfügen Phi-Funktion]

setzen entsprechender Operand

commit join-Knoten

Werte Zuweisungen von ϕ aus

Sonderfall ~~while~~ ~~repeat~~ ϕ var Schleifenkörper
umbenennen in Schleifenkörper

Sonderfall ~~repeat~~ aktueller Wert am Ende
nicht von ϕ , da nicht durch
join Knoten verlassen

$v_i = \phi(\dots v_n)$

v_n statt v_i

Sicherung
 $\downarrow$

$$a_2 = \phi(a_1, \dots) \mid a_0$$

Rückwandlung entfernen ϕ

Nummern einfach vergessen nach Optimierungen nicht möglich

$x_3 = \phi(x_1, x_2)$ Kopieranweisung in pred $x_3 := x_1$ $x_3 := x_2$

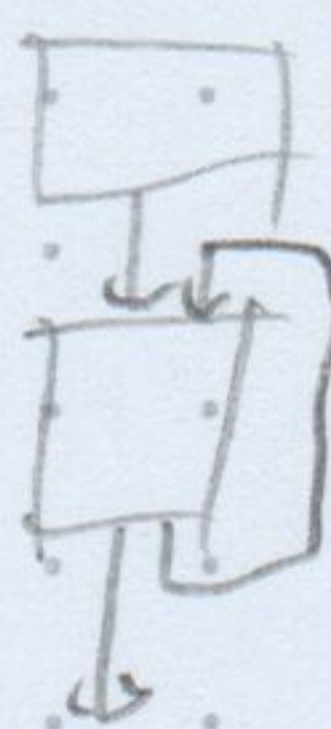
! Kritische Kontrollflusskante



kopieren in pred teilw. unnötig

$\Rightarrow$ Kante aufspalten - zusätzliche Sprunganweisung

Lost-copy Problem



nicht aufspalten
kritischer Kante

! schreiben in
Lebenszeit

$\Rightarrow$ Sicherheitskopie
blockübergreifend

! auch in condition
(lokal)

Fehlerhafte Verkürzung

formal ϕ -Funktionen Parallel

Konfliktgraph $\rightarrow$ liest verschreibl

$\Rightarrow$ lokale Kopie, topologisch sortieren

Algorithmus nach Briggs, ...

rekursiv durch CFG Pre-Order

HashMap original var $\rightarrow$ stack Kopien

Vernichtung durch letzte Kopie ersetzen

in pushed Menge original merken um bei Verlassen
Lebensbereich wieder runterzunehmen

Flag: isPhiParam [v]

alle nötigen Kopien für succs in copyset

worklist mit isPhiParam [dest] = false

! in cond $\Rightarrow$ lokale Kopie

! in b live Out $\Rightarrow$ globale Kopie

konflikt $\Rightarrow$ lokale Kopie

Frei gewordene in worklist

Optimierung wie Erfahrener Assembler entwickelt
Abstraktion effizient auf (schwachen) Ziel abbilden

Geschwindigkeit

Größe

Speicherzugriffe

Speicherseiten

Sicherheit - Gleiche Semantik

Profitabilität - ausreichende Verbesserung

Risiko - negative Effekte

Anwendbarkeit - ausreichend viele Stellen

Lokal - nur BB

Superlokal

auf Extended Basic Block $(b_1, \dots, b_n)$ b_i ist genau 1 Vorgänger
bilden Baum

Lokale Methode auf Pfad

statt Neuberechnen Rücksetzen nach Geltungsbereich

Regional über Merge-Points hinweg

! zusammenführen Rechenintensiv

⇒ nur gemeinsames Pfad-Präfix nutzen (Dominator
SSA sonst Probleme Zuweisungen)

Präfix Reihenfolge auf Dominatorbaum

Global

allgemein analyse
transform

Schlüsselloch / peephole Optimierung
ersetze pattern z.B. auf Assembler

Tail recursion Stack Frame behalten

Redundanter Ausdruck - vorher berechnet, Operanden unverändert

SSA, Ast, hashen

Local Value Numbering

- Arrays,
Zeiger,
Faktoren,...

+ Wiederverwenden
- Lebenszeiten

Hashe vergleichen

! Koll. in ienen $\Rightarrow$ genauer Vergleich
einfache Ausdrücke $\Rightarrow$ Kollisionen vermeidbar

- Aufwand
pro Operation
- nur lokal

Bestimme Wertnummern Operanden

Hash bekannt?

{ Wertnummer eintragen für erg.
neue Wertnummer Expr. und erg.

Constant Folding

berücksichtigen abg. Eigenschaften kommutativ, $+0$, $+0$,...

0 vor Definition, hochzählen mit jeder Def.

Ausdruck	auf Wertnummern	Wertnummer	Expr
a	-	1	$a^1 b^2 c^3$
b	-	2	
a+b	$1+2$	3	
c	-	3	

SSA für Umschreiben

in

Wie

Superlokales VR

in

DRNT - regional

#

Global Common Subexpr. Elimination

kein SSA, Wertnummern,
Lexikalische Gleichheit

lokal - Wertgleichheit
global - lexikalisch

Datenflussanalyse

Expr. definiert, wo berechnet

kill, wo operand def.

verfügbar Vpfade nicht gelöschte Definitionsstelle
 $\Rightarrow$ nicht neu berechnen, evtl. in tempor. Kopieren

$$Avail(b) = \bigcap_{Pred} DEExpr(b) \cup (Avail(a) \cap \overline{ExprKill(a)})$$

↓
downwards exposed

globale Bezeichner für alle Expr in Avail

lokales VR, initialisiert mit Avail(b)

Versionnummer 0

ersetze und merke Wiederverwendung $USE[Expr]$
einfügen nötiger Kopien an Blockende

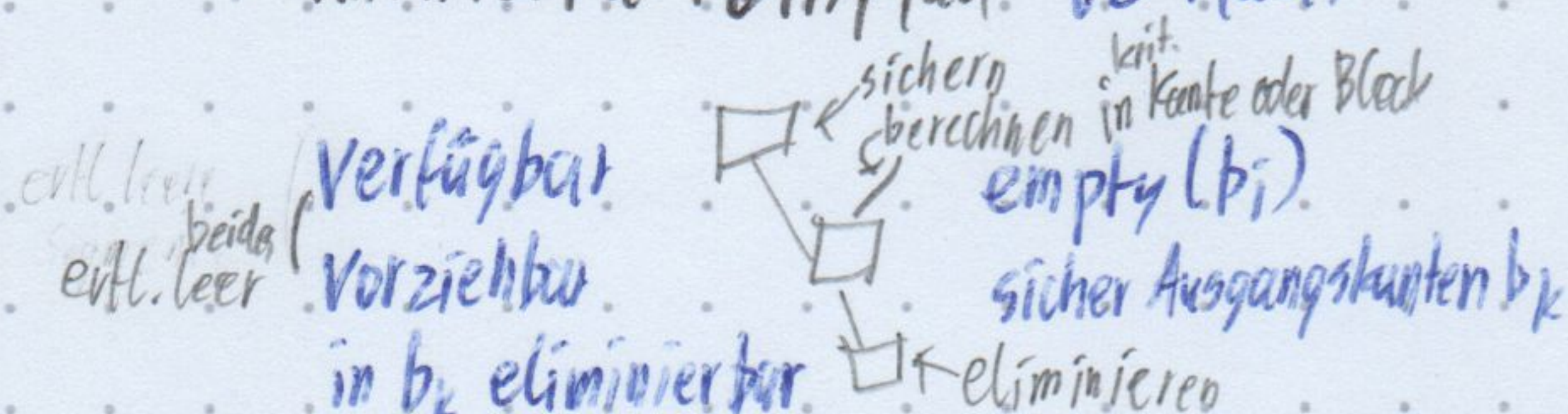
Total
Partielle Redundanz: Fall auf dem schon berechnet

z.B. Loop Invariant Code Motion muss Verwendung, Ausgäingedominieren

Partial Redundancy Elimination (PRE)

Optimale Lebenszeit von Neuberechnung
Berechnungsoptimalität

Eliminierbarkeitsspfad (E-Pfad)



Prädikaten Schreibweise
rechnen mit Vektor aller exprs
Bifunktor

comp - lokal verfügbar
downward exposed expr
antloc - lokal vorziehbar
upward exposed expr
transp - keine Zuneigung Operanden
¬expr.kill

availin = Πp availant
availout = availin · transp + comp

ant in = ant out · transp + antloc
ant out = Πs ant in
vorziehbar / anticipated
~~anticipatable~~

Finde E-Pfad ^{Start} Suffix, rückwärts nach Startblock E-Pfad
 $\tau_{avail}(b_m) \cdot ant(b_m) \cdot \epsilon_{parail}(p)$
Antizipated / vorziehbar

$$eps_{in_e}(b) = (\epsilon_p(avail_{out_e}(p) + eps_{out_e}(p))) \cdot ant_{in_e}(b) \cdot \tau_{antloc}(b)$$

$$eps_{out_e}(b) = eps_{in_e}(b) \cdot \tau_{antloc}(b)$$

Local vorziehbar

$$redundant_e(b) = (eps_{in_e}(b) + \overline{avail_{in_e}(b)}) \cdot \overline{ant_{loc_e}(b)}$$

Ent block Pfad End block suffix

$$insert_e(b_n) = \tau_{avail_{out_e}(b_n)} \cdot \tau_{eps_{out_e}(b_n)} \cdot \Pi s eps_{in_e}(s)$$

krit. Kante - $insert_e(b_n, b_j) = \Pi s \cdot \tau_{insert_e}(b_n) \cdot eps_{in_e}(b_j)$

nicht verfügbar; mündet in E-Pfad-Suffix

$$sv_{up_{out_e}(b)} = \{ \epsilon_s(eps_{in} + redund + sv_{up_{in}}) \cdot avail_{out}$$

hier oder drüben
sicherung möglich

$$pred \text{ muss sichern} = sv_{up_{in_e}(b)} = sv_{up_{out}} \cdot \tau_{comp}$$

sicherung nicht selbst möglich

$$save_e(b) = sv_{up_{out}} \cdot comp \cdot \tau(redund \cdot transp)$$

gebraucht möglich

⇒ redundant, insert, save

Skalare Optimierung

Dead Code Elimination Dead

nutzloser Code - Erg. nicht verwend. /
unerreichbar - CFG Pfad

kritische Operationen - außen sichtbare Effekte
return, var Parameter, globale Var, calls, IO

Mark & Sweep - Operationen, verbiege unnutzbare Verzweigung

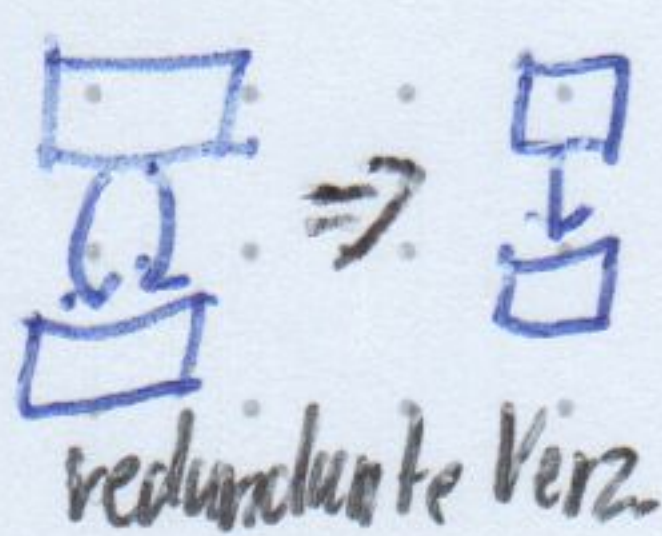
init krit. Operationen

alle benötigten und entscheidende Verzweigung

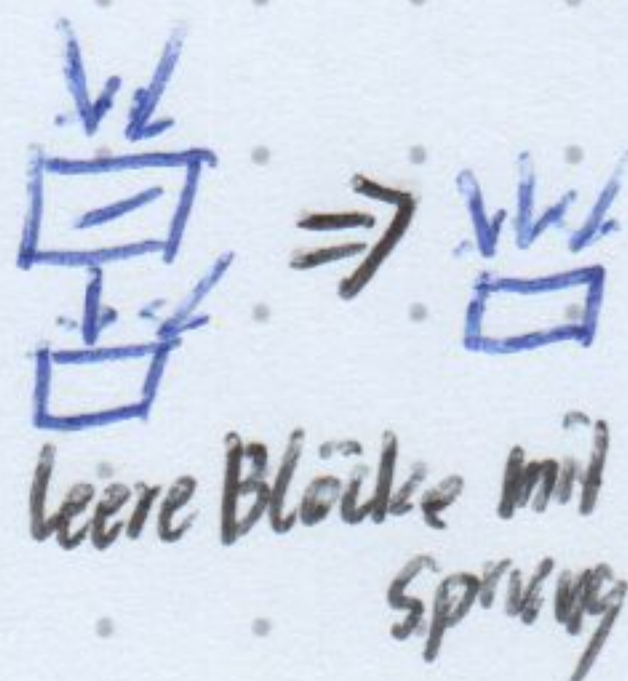
↳ kontrollabhängig (DF reverse CFG)

~~CFE~~

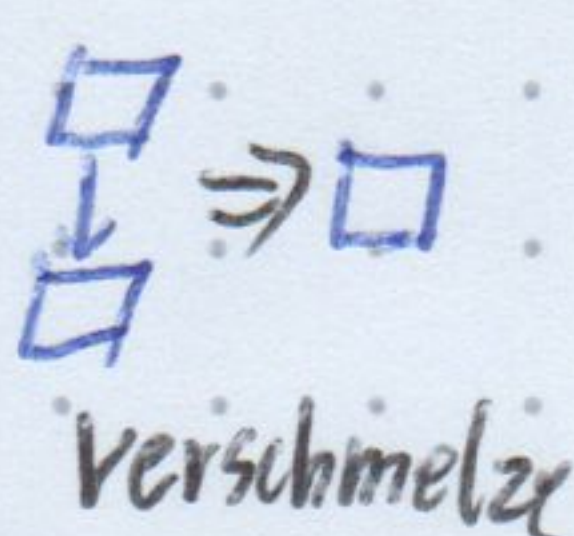
Clean



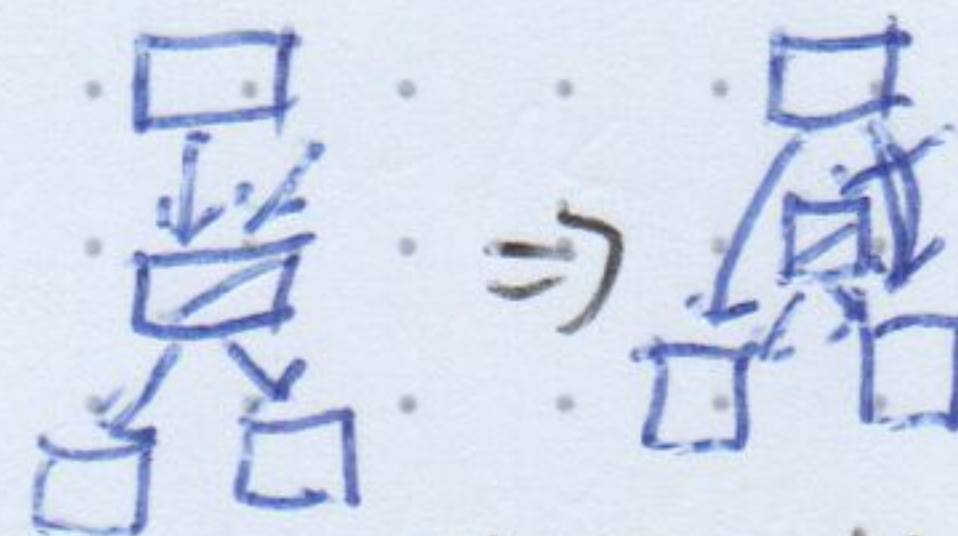
redundante Verz.



leere Blöcke mit Sprung



Verschmelzer



Heranziehen von Verzweigung

Postorder bis sich nichts
iterativ

kann ohne Dead. keine Schleife beseitigen



! Entfernen Entlosschleife

in (11) legal wenn cond. nicht konst. Expr.

Sparse Simple Constant Propagation SSC

Verband

T undef, potential const.

c_i

$\perp$ not-a-const.

~~SSA~~ x op y

$| c_1, c_2 \Rightarrow c_1 \text{ op } c_2$

$| +, -, \dots \Rightarrow \perp$

$| \dots \Rightarrow T$

optimistische meets an join Knoten ϕ

$| T, c_1 \Rightarrow c_1$

$| c_1, c_2 \Rightarrow c_1 = c_2 ? c_1 : \perp$ // definitiv unterschiedlich

Worklist von SSA-Graph Def $\rightarrow$ Use Kanten

arith. Besonderheiten nutzen

Sparse Conditional Constant Propagation SCC

SSA Worklist - Def $\rightarrow$ Use

CFG Worklist - erreichbare Blöcke
Flag damit je nur einmal

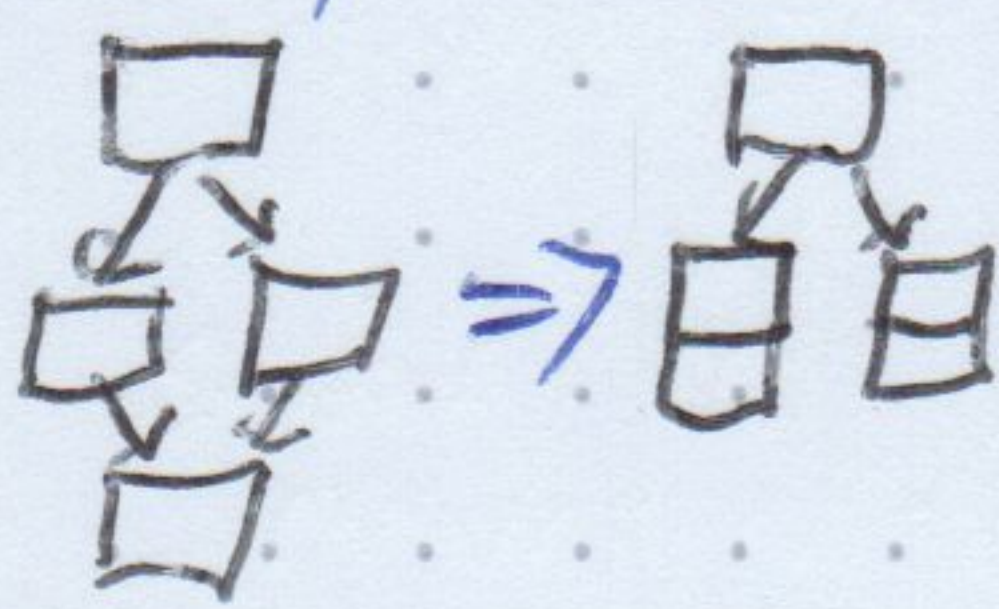
BB wird erreichbar durch $c_i, \perp$ branch, jump

bleibt T in Sprungbed. $\Rightarrow$ Comp. Fehler

unerreichbare Pfade tragen T zu ϕ bei

Optimierungsermöglichende Transformationen Code oft größer, aber mehr Angriffsfläche

Cloning



+ längere BB
weniger Verzweigung
- 1\$-Misses

Inlining Prozeduren

+ längere Prozedur
keine Aufrufsequenz
- 1\$-Misses
Registernachfrage

Loop Unrolling

z.B. 4 Schleifeniterationen
zusammenlegen

n unbekannt

1 vorherige Schleife für letzte Iterationen hinterlegen

1 $n \% 2 > 0$ 1 Iteration

$n \% 4 > 1$ 2 "

...

+ Zahl ausgeführte Anweisungen

Anw. in Rumpf, parallel?, aufeinanderfolgende Speicherzugriffe

Branch-Delay-Slots

- 1\$-Misses

Loop Unswitching

statisches if aus
Schleife rausziehen

+ weniger Kontrollfluß
in Schleife

Optimierung

Kleinerer Code

Code Hoisting - Very Busy Expr. nur einmal

Sinking - wiederkehrende Anweisungsfolgen nach hinten

Cross Jumping - Anweisungen hinter Label

Procedure Abstr. - wiederkehrende Anweisung

- Aufrufsequenz

- verlängert

Vermeiden 1\$-Misses, Page-Faults

Procedure Placement - häufige in Nähe Aufrufer

Block Placement - häufigerer Branch als Fall-Through
+ Prefetching z.B. durch Profiler

Fluff Removal - seltener Code an Rand Programm

Optimierungsreihenfolge

Interaktionen

verstärkend

verschlechternd

überlappend

Instruction Selection → Register Allocation → Instruction Scheduling → Machine Code

in SSA schneller
aber auflösen
braucht Register

k Maschineregister
vermeide spills/splitting
Laufzeiteffizienz max $O(n^2) \Rightarrow$ heuristisch
np-vollständig

Allocation - Welche Werte?
Zuweisung - Wo?

Aufwand
dynamisch - #ausgeführte
statisch - #gespeicherte

ILX - Intermediate Language for Optimizing Compilers

virtuelle Register

loadl 2 $\Rightarrow$ rr 1
loadhl rr0 @offset $\Rightarrow$ vr 1
add vr1, vr2 $\Rightarrow$ vr 0
nur Def, Use, relevant

Lebenszeit / live range - letzte def - letzte use
in BB Intervall

lebt nur bis Zeile von Use
lesen erfolgt vor schreiben in Instr. (nicht lesen)
 $\Rightarrow$ annotation hinter Instr. entspricht nach Instr.

MaxLive max #Lebenszeiten, die B. enthalten

$\leq k \Rightarrow$ einfach
 $> k \Rightarrow$ F Register für Spills nötig
Berech. Speicheraddr. $\sim$ Ziel-
geladener Wert Architektur

Top-Down Allocator

Alloziere k-F wichtigste Register
Heuristik
z.B. Häufigkeit Auftreten
- vr 1 tritt z.B. nur oft am
Anfang & Ende auf

iterativ
Verschiebe einen Wert mit niedrigster prio in Speicher
spill vr 1 $\Rightarrow$ 16
restore 16 $\Rightarrow$ vr 1 trennt Lebenszeit

Bottom-Up Allocator $\sim$ Paging
bald wiedergebrauchte in Register
bei Bedarf in Register erh. freigeben
bevorzugt dirty

Für jede Registerklasse $class(rr_x)$, die rr_x aufnehmen könnte
Name[] // Bsp.
Next[] // used
Free[] // flag
[Dirty[]]

$vr1 \leftarrow vr2 \text{ op } vr3$
 $rx = ensure(vr2)$
 $ry = ensure(vr3)$ find, sonst allocate
if rx, ry not needed after $\Rightarrow$ free $\sim$ Attr. resetten auf stack // möglicherweise für rz
 $rz = allocate(vr1)$ - pop from stack | find Max Next (save if dirty)
set Attr. Next := -1 damit nicht wieder freigegeben

+ inkasiv
so viel nützlich wie möglich
vorladen

bevorzugung dirty
- ineffizient wenn zu lange

setze Next von ~~rx~~ [rx, ry] rz

Globale Registerallokation

Graphenfärbungsproblem auf Konfliktgraph

Lebenszeiten in CFG
 $\text{Kdefs } E' \Rightarrow \text{Kuses } E''$
 $\text{Kuses } E' \Rightarrow \text{Kdefs } E''$
Variable kann mehrere Lebenszeiten haben

Lebenszeiten in SSA

fließen an ϕ zusammen

Umbenennen Var + Lebenszeiten von Var

Datenflussanalyse rückwärts U Live In Out berechnen

Inferenzgraph

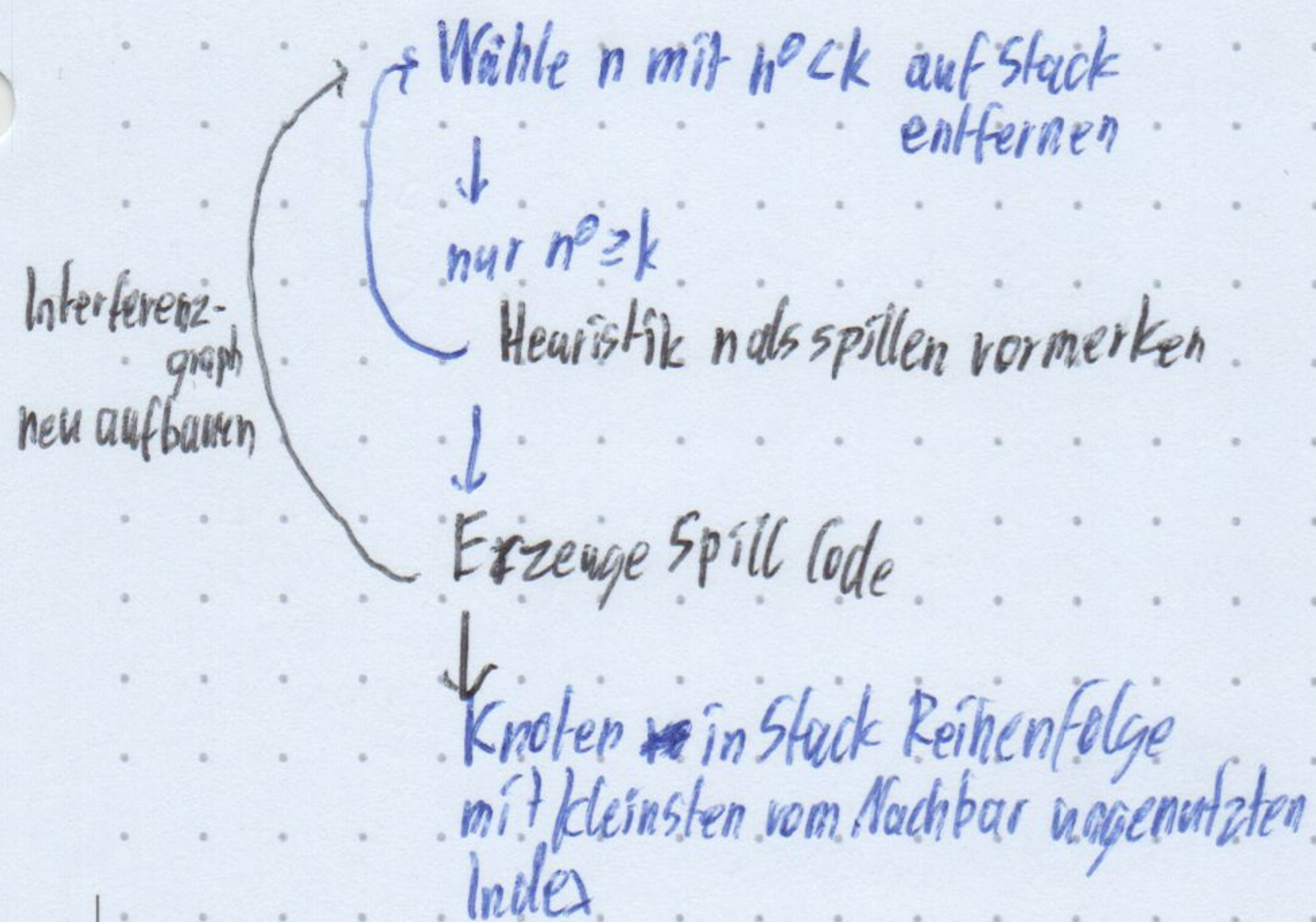
Lebenszeit ~ Node

überschneiden ~ Konflikt

Grad Knoten n^0

$n^0 < k \Rightarrow$ erfolgreicheinfärbbar

Chaitins Algorithmus



pessimistisch bei $n^0 \geq k$

$\Rightarrow$ Briggs notwendigkeit spillen erst bei runternehmen von Stack

löst z.B.
 $k \geq 2$



LLVM - Compiler Infrastruktur Projekt

Open Source viele Sponsoren Hardware Firmen Software Firmen

LLVM Core Analyse, Optimierung, Codeerzeugung
Clang C Frontend
MLIR Multilayer IR
libc++

Entkopplung
sehr modular
hackable
Frontend
Optimierung
Codeerzeugung

eigenständige Darstellungsformen

clang → opt → llc

textuell (Assembler.ll)
binär (.bc)
C++ Objekte

clang

Ast durch rekursiven Abstieg, Visitor
Decl, stmt, type
Expr
"Makromonster"

LLVM-IR SSA-Form Low-level

@identifizier - global var % - lokal

@c = global i32 42 int c = 42;

define i32 @f(i32 %a, i32 %b) {

entry:

%c = add i32 %a, %b

%0 = load i32, ptr @c

ret i32 %c

}

i1

i32

half

float

ptr

[n x Type] Array

{type, ...} Struktur

<type, type...> Vektor

nuw - no unsigned wrap
nsw - signed

br i1 %cond, label %if.then, label %if.retr br label %...

if.then:

if.else:

select i1 %cond, i32 %a, i32 %b

%x = phi i32 [%x1, %if.then], [%x2, %if.else.]

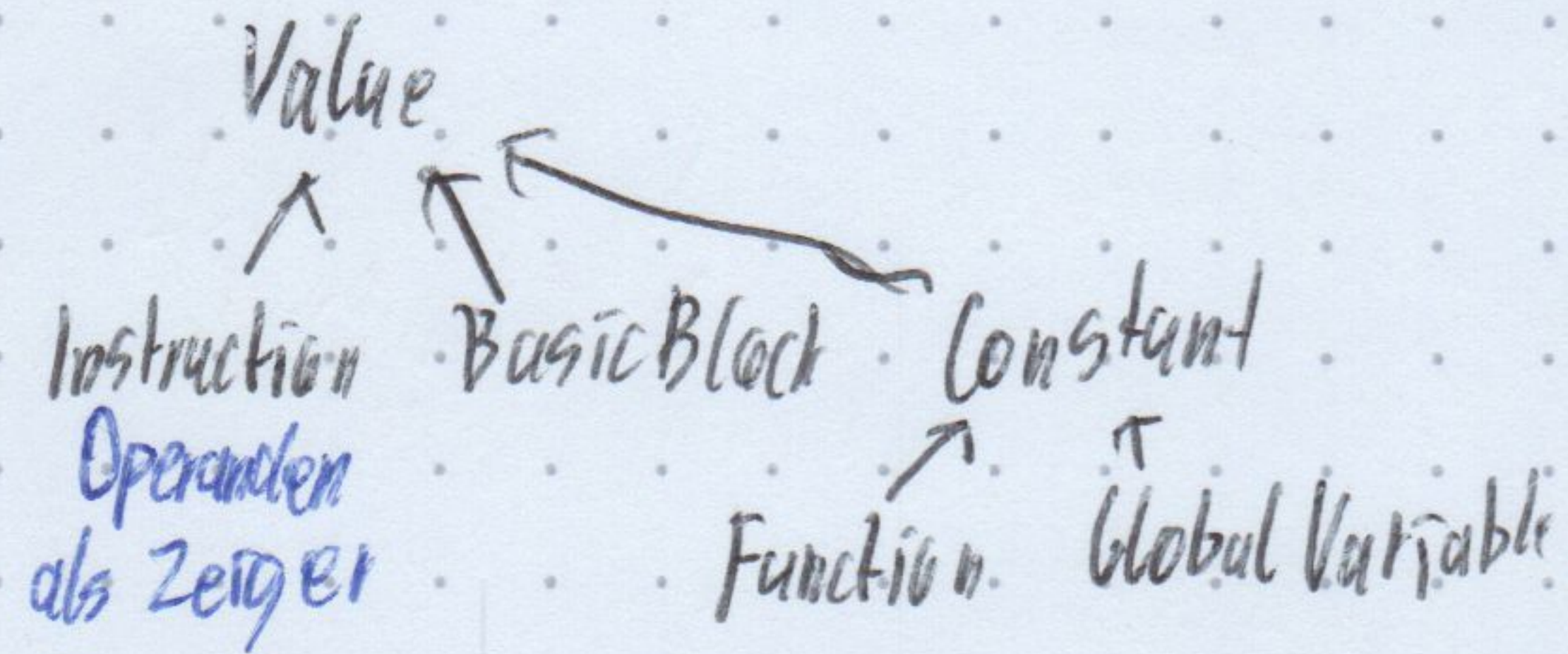
call i32 @f(i32 1)

getelementptr <ty>, ptr <ptrval>, <ty> <idx>*

<ty> of
Pointer

meist. recursive rein offset
i32 beginnend mit type.ptr.
erstes oft 0. to piece z.B. Array

LLVM-IR API



getOperand(i);
getCondition();
getArgOperand(i);
getCalledFunction();

getEntryBlock();
pred_begin(BB) succ. "
red_end(BB) succ. "

```

for (Function RF : M) {
  for (BasicBlock BB : F) {
    for (Instruction I : BB) {

```

getParent hohe Ebene

uses();
use.getUse();

Visiter

Codeerzeugung Target-Independent Code Generator
targets: RISC-V, ... architekturneutrale Algorithmen

Viele tablegen-Format(*.td) ~ Records
Beschreibungen Abstraktion

LLVM-IR → Selection DAG → Machine IR → MC Layer

Graph Data Flow in BB Scheduler
chain für Ordnung virtuelle Register
Unsupported und Extensions Optimierung
of Op möglich Routinenprotokoll
Registerallokation

Typ-Legalisierung Op-Library custom lowering
nur unterstützte Types
lokale Optimierungen

Instruction Selection
SDNodes → Bpcodes
aus Tablegen